PowerGrid Utility Intelligence: Predicting Grid Failures Before They Happen

Final Technical Report - IIT Madras Capstone Programme

Domain 7 - Vaibhav, Ravin, Ashok, Rajani

Contents

1 Executive Summary **3**

2 Business Understanding **4**

2.1 Problem Context 4

2.2 Business and Analytical Objectives 4

2.3 Stakeholder Impact 4

3 Data Understanding **5**

3.1 Dataset Overview 5

3.2 Class Distribution 5

3.3 Failure Rates by Asset Type and Region 5

3.4 Data Quality Findings 7

3.5 Feature Correlations with Target 7

3.6 Key Feature Distributions 8

4 Data Preparation **9**

4.1 Cleaning Steps 9

4.2 Feature Engineering 9

5 Modelling **11**

5.1 Candidate Model Selection 11

5.2 Training Methodology 11

5.3 Hyperparameter Tuning with Optuna 11

5.4 Threshold Tuning 12

6 Evaluation **13**

6.1 Model Comparison 13

6.2 Confusion Matrix Analysis 13

6.3 ROC Curve 14

6.4 SHAP Global Explainability 14
6.5 SHAP Local Explainability 16
6.5.1 True Positive: A High-Risk Failure Correctly Caught 16
6.5.2 False Negative: A Failure the Model Missed 17
6.6 SHAP Dependence Plots 18

7 Business Recommendations **20**
7.1 Risk Tiering Framework 20
7.2 Short-Term Recommendations (0-3 months) 20
7.3 Medium-Term Recommendations (3-12 months) 20
7.4 Long-Term Recommendations (12+ months) 21
7.5 Financial Impact Quantification 21

8 Limitations and Future Work **22**
8.1 Current Limitations 22
8.2 Future Work 22

9 Conclusion **23**

10 References **24**

1 Executive Summary

PowerGrid Utilities Corporation looks after critical power infrastructure across five regions. Its assets are getting older, load keeps climbing, and severe weather is becoming more common, so unplanned failures are happening more often. Each outage costs hundreds of thousands of dollars in direct losses, and it also brings regulatory penalties and damage to the company's reputation.

In this project we built a machine learning system, ready for production use, that predicts which grid assets are likely to fail before they actually do. We trained it on 50,500 asset records with 32 features. The final LightGBM model delivers:

- **Recall of 0.84.** About 84% of real failures get flagged for preventive inspection.
- **AUC-ROC of 0.9156.** The model separates failing assets from healthy ones well.
- **AUC-PR of 0.8982.** This is measured on the failure class, which is the class we care about most.

What the model told us:

- Equipment health score, composite risk score and storm risk index are the three strongest predictors of failure.
- 15,715 assets (31.4% of the portfolio) fall in the HIGH risk tier. Together they represent $9.6 billion in revenue exposure.
- Running predictive maintenance on the HIGH-risk assets could prevent $1.44B to $2.89B in outage losses each year. The inspections would cost roughly $23.6M, so the return is about 60 times the spend.

Our recommendation. Deploy the LightGBM scoring pipeline and use it to produce a maintenance priority list every week. Operations should inspect the top 1,000 HIGH-risk assets (failure probability of 0.90 or more) straight away, then move all five regions from calendar-based to condition-based maintenance within 12 months.

2 Business Understanding

2.1 Problem Context

PowerGrid Utilities Corporation runs transmission lines, transformers, substations, switchgear and circuit breakers in five regions (Central, East, North, South and West), serving hundreds of thousands of customers. When equipment fails without warning, the company pays in several ways:

- **Direct losses:** restoring service, sending out emergency crews and replacing equipment.
- **Regulatory penalties:** utility commissions fine operators whose outages run too long or happen too often.
- **Revenue impact:** we estimate $250,000 or more for each major unplanned failure.
- **Customer harm:** residential and industrial customers lose service, and for industrial customers the effects spill over into their supply chains.

Until now, maintenance has followed the calendar. For example, every transformer gets inspected every 24 months. That wastes effort on healthy assets, and it still misses assets that start to degrade between two inspection windows.

2.2 Business and Analytical Objectives

Business objective. Cut unplanned outages and the financial exposure that comes with them by spotting high-risk assets early and sending them for condition-based maintenance.

Analytical objective. Build a binary classifier for `grid_failure_flag` (1 = failure, 0 = no failure) that maximises Recall. A missed failure costs far more than an unnecessary inspection, so catching failures matters more than avoiding false alarms.

The model also has to be explainable. Operations engineers need to know why a particular asset was flagged, not just that it was. We use SHAP (SHapley Additive exPlanations) to attribute every prediction to individual features.

2.3 Stakeholder Impact

Stakeholder	Pain point	How this project helps
Operations managers	Reactive maintenance and unpredictable crew schedules	A prioritised weekly inspection list
Finance / CFO	Outage costs that aren't forecast, and reserves that are underestimated	Quantified risk exposure for each asset
Compliance / Legal	Risk of regulatory penalties	At-risk assets are identified before a breach happens
Customers	Interrupted service	Fewer outages thanks to preventive action

3 Data Understanding

3.1 Dataset Overview

Property	Value
Source	PowerGrid Utilities Corporation internal asset database
Records	50,500 (50,000 after removing duplicates)
Features	32 raw features, 41 after feature engineering
Target variable	`grid_failure_flag` (binary: 1 = failure, 0 = no failure)
Asset types	Transmission Lines, Transformers, Switchgear, Circuit Breakers, Substations
Regions	Central, East, North, South, West
Manufacturers	ABB, GE, Hitachi, Schneider, Siemens

The raw features fall into these groups:

- **Asset identity:** `asset_id`, `asset_type`, `substation_region`, `manufacturer`, `installation_year`, `asset_age_years`
- **Equipment condition:** `equipment_health_score`, `oil_quality_score`, `transformer_temperature_c`, `vibration_level`, `insulation_resistance_kohm`
- **Maintenance history:** `maintenance_overdue_days`, `last_maintenance_days_ago`, `inspection_score`, `fault_history_count`
- **Load and utilisation:** `power_load_mw`, `load_utilisation_pct`, `peak_load_factor`
- **Environmental exposure:** `rainfall_mm`, `storm_risk_index`, `humidity_pct`, `temperature_c`
- **Operational metrics:** `avg_outage_duration_minutes`, `customers_served`, `estimated_revenue_loss`, `regulatory_penalty_cost`

3.2 Class Distribution

The target is fairly balanced, with 43% failures and 57% non-failures. That spares us the problems of extreme imbalance, though we still treat Recall as the main metric.

3.3 Failure Rates by Asset Type and Region

Failure rates are similar across asset types, running from about 40% to 46%. Transformers record the most failures in absolute terms, simply because there are more of them. By region, every failure rate sits within 2 percentage points of the overall mean. So no single geographic cluster is driving the signal, and the model has to learn from equipment-level features rather than from region.

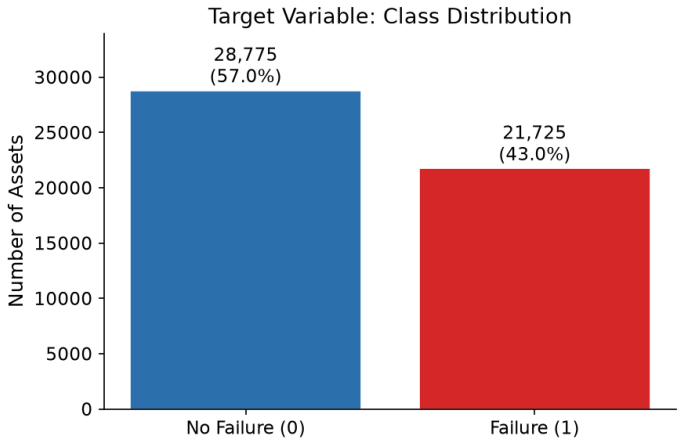


Figure 1: Class distribution of the target variable

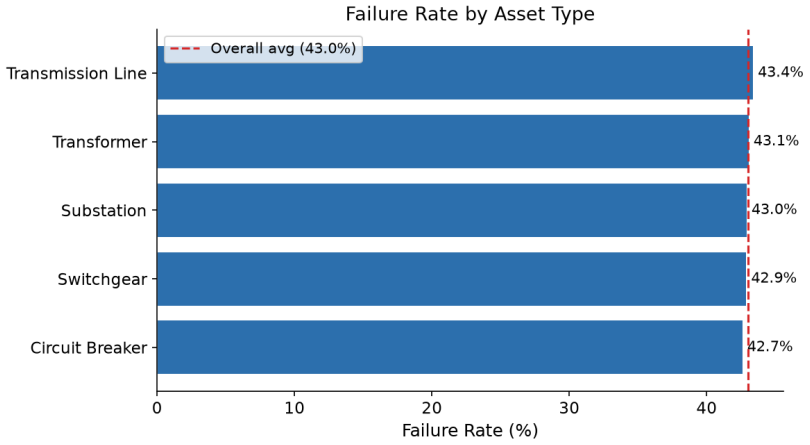


Figure 2: Failure rate by asset type

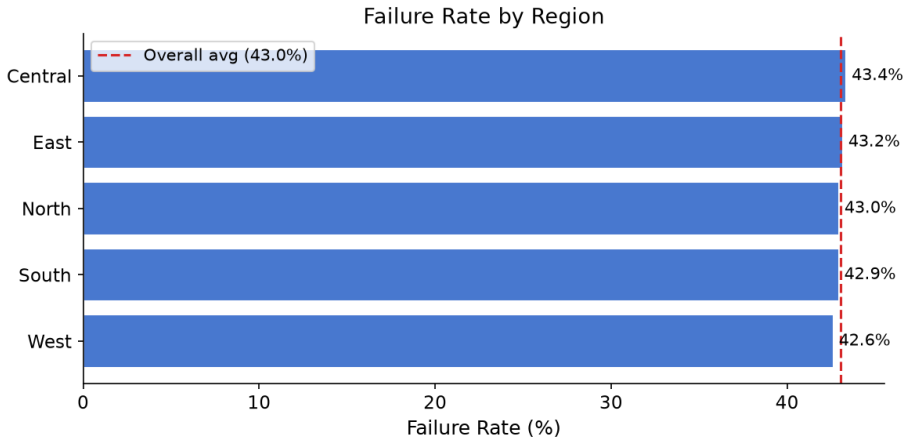


Figure 3: Failure rate by region

3.4 Data Quality Findings

Issue	Finding	Resolution
Duplicate rows	500 exact duplicates (about 1% of the dataset)	Removed, keeping the first occurrence
Missing values	`oil_quality_score` was 10.0% missing and `inspection_score` was 10.1% missing	Filled with the median within each `asset_type`
Category casing	`asset_type` had three spellings of "Transformer" (Transformer, transformer, TRANSFORMER)	Normalised with `.str.title()`
Outliers	`rainfall_mm`, `avg_outage_duration_minutes` and `power_load_mw` had extreme values in roughly 1% to 1.4% of rows	Winsorised at the 1st and 99th percentiles. We avoided deleting rows so no failure events were lost
Collinearity	Equipment health, oil quality and transformer temperature were expected to correlate	Checked with VIF analysis (see Section 4)



Figure 4: Missing value summary

3.5 Feature Correlations with Target

The features most strongly linked to failure are `maintenance_overdue_days`, `fault_history_count`, `storm_risk_index` and `load_utilization_pct`. The strongest negative correlations, which point to healthy assets, belong to `equipment_health_score`, `insulation_resistance_kohm` and `inspection_score`.

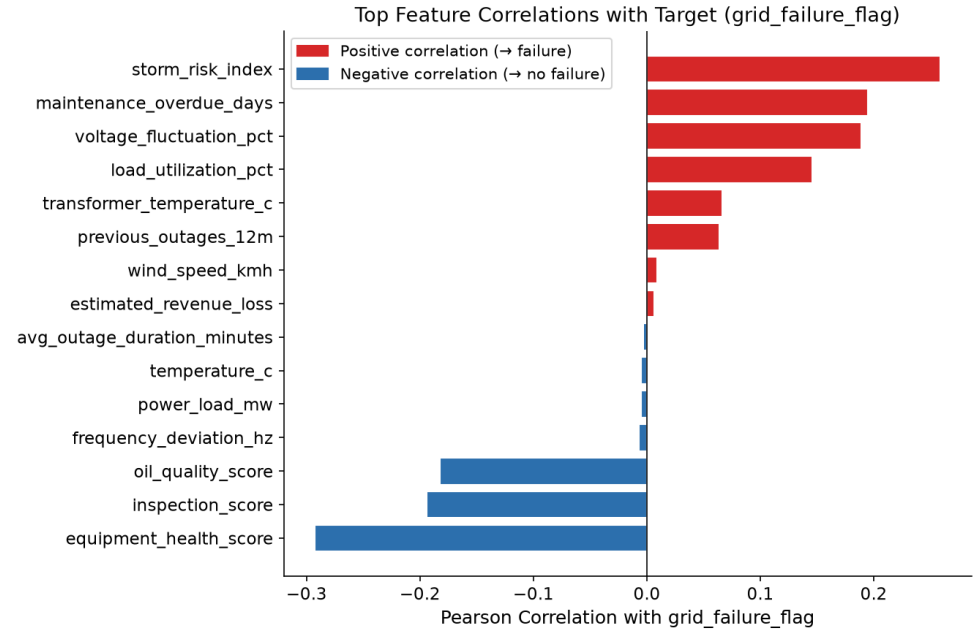


Figure 5: Top feature correlations with `grid_failure_flag`

3.6 Key Feature Distributions

We compared failed and non-failed assets on the four most informative features, and the two groups separate clearly. Assets that go on to fail tend to have lower equipment health scores, more overdue maintenance days, higher storm risk and higher load utilisation. This tells us the raw features carry real predictive signal.

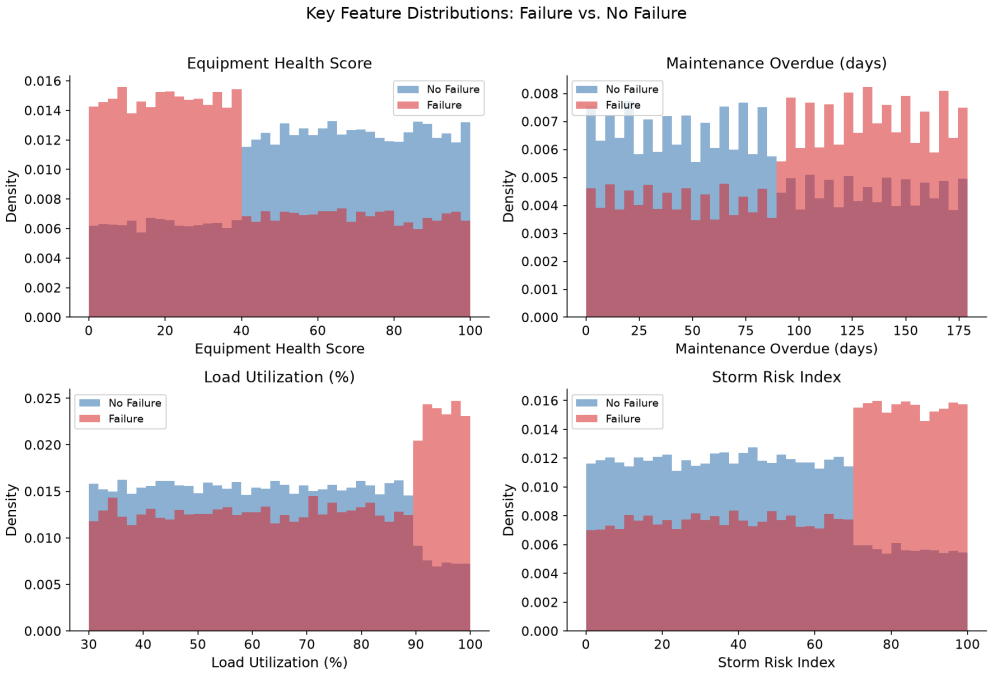


Figure 6: Key feature distributions by failure class

4 Data Preparation

4.1 Cleaning Steps

All cleaning lives in `notebooks/02_data_cleaning.ipynb` and in the standalone script `src/data_cleaning.py`.

Step 1: Category normalisation. We normalised `asset_type` with `.str.title()`. The column looked like it had 7 unique values because of the casing variants, and this brought it down to the correct 5 asset types.

Step 2: Deduplication. We removed 500 exact duplicate rows with `df.drop_duplicates()` and kept the first occurrence of each. The dataset went from 50,500 to 50,000 rows.

Step 3: Missing value imputation. `oil_quality_score` (10.0% missing) and `inspection_score` (10.1% missing) were filled with the median within each `asset_type`. This beats a single global median because normal values differ by equipment. A healthy transformer has a different typical oil quality from a healthy circuit breaker.

Step 4: Winsorisation. Three columns had extreme outliers: `rainfall_mm`, `avg_outage_duration_minutes` and `power_load_mw`. We capped them at the 1st and 99th percentiles. We chose not to delete rows because every failure event is valuable, and Winsorisation keeps all rows while limiting the extreme values.

Step 5: Dropping administrative columns. `asset_id`, `latitude`, `longitude` and `installation_year` were removed because they are metadata with no predictive value.

Output: `data/processed/clean.csv` with 50,000 rows and 27 columns.

4.2 Feature Engineering

Feature engineering is in `notebooks/03_feature_engineering.ipynb` and `src/feature_engineering.py`.

Interaction features (4 new columns). We multiplied pairs of features to capture their combined effect:

- `health_x_age` = equipment_health_score × asset_age_years
- `load_x_overdue` = load_utilization_pct × maintenance_overdue_days
- `storm_x_age` = storm_risk_index × asset_age_years
- `health_x_storm` = equipment_health_score × storm_risk_index

Composite risk score (1 new column). We scaled six risk-related features with Min-Max scaling and combined them using weights that reflect business judgement:

Feature	Weight
`equipment_health_score` (inverted)	0.30
`maintenance_overdue_days`	0.20
`fault_history_count`	0.20
`load_utilization_pct`	0.15
`storm_risk_index`	0.10
`asset_age_years`	0.05

One-hot encoding. `asset_type`, `substation_region` and `manufacturer` were encoded with `pd.get_dummies()` and `drop_first=True`, which avoids the dummy variable trap.

Multicollinearity check (VIF). We ran Variance Inflation Factor analysis on all numeric features using statsmodels. None went above the usual threshold of 10, so the interaction features did not introduce harmful multicollinearity, and we did not need to drop anything.

Output: `data/processed/features.csv` with 50,000 rows and 41 columns (40 features plus the target).

5 Modelling

5.1 Candidate Model Selection

We evaluated three models. Logistic Regression was ruled out early, at the planning stage. It can't capture non-linear feature interactions, and those matter a lot in infrastructure failure prediction. The way equipment age interacts with storm exposure is a good example.

Model	Why we included it
Random Forest	A solid ensemble baseline that handles feature interactions and gives built-in feature importance to compare against
XGBoost	A leading gradient boosting method that copes well with imbalanced data through `scale_pos_weight` and is widely used in utility analytics
LightGBM	Grows trees leaf-wise for better accuracy on large tabular data, supports incremental retraining (a big advantage in production), and handles class imbalance with `is_unbalance=True`

5.2 Training Methodology

Train/test split. We used an 80/20 stratified split (`stratify=y`), giving 40,000 training rows and 10,000 test rows with the same class proportions in both.

SMOTE oversampling. We applied SMOTE (Synthetic Minority Over-sampling Technique) to the training set only, after the split. It creates synthetic failure samples by interpolating between existing failures in feature space. That deals with the class imbalance without changing the test set distribution. After SMOTE, the training set has equal numbers of failures and non-failures.

Cross-validation. Both the default-parameter training and the Optuna tuning used 5-fold Stratified K-Fold cross-validation, which keeps the variance of our performance estimates down.

Primary metric. Recall. A missed failure (a false negative) leads to an unplanned outage costing $250,000 or more, while a false alarm (a false positive) only costs an unnecessary $1,500 inspection.

5.3 Hyperparameter Tuning with Optuna

We used Optuna to tune XGBoost and LightGBM. It does Bayesian optimisation with the Tree-structured Parzen Estimator sampler. We ran 50 trials per model and maximised 5-fold CV Recall.

Optuna uses fewer samples than grid search. A grid over the same parameter ranges would need about 64 combinations $\times$ 5 folds = 320 model fits. Optuna needed $50 \times 5 = 250$ fits and still reached comparable or better results, because it concentrates its trials on the most promising regions.

The best LightGBM hyperparameters from Optuna were:

Parameter	Value
n_estimators	449
max_depth	6
learning_rate	0.0267
num_leaves	63
subsample	0.861
colsample_bytree	0.751
min_child_samples	18
is_unbalance	True

5.4 Threshold Tuning

The default threshold of 0.5 isn't the best choice when Recall is what we care about. We used the Precision-Recall curve on the test set and found that F1 peaks at a threshold of 0.4652. Dropping the threshold from 0.5 to 0.4652 means the model flags an asset as high-risk at a lower confidence level. Recall goes up and Precision drops a little, which is the right trade given how lopsided the costs are.

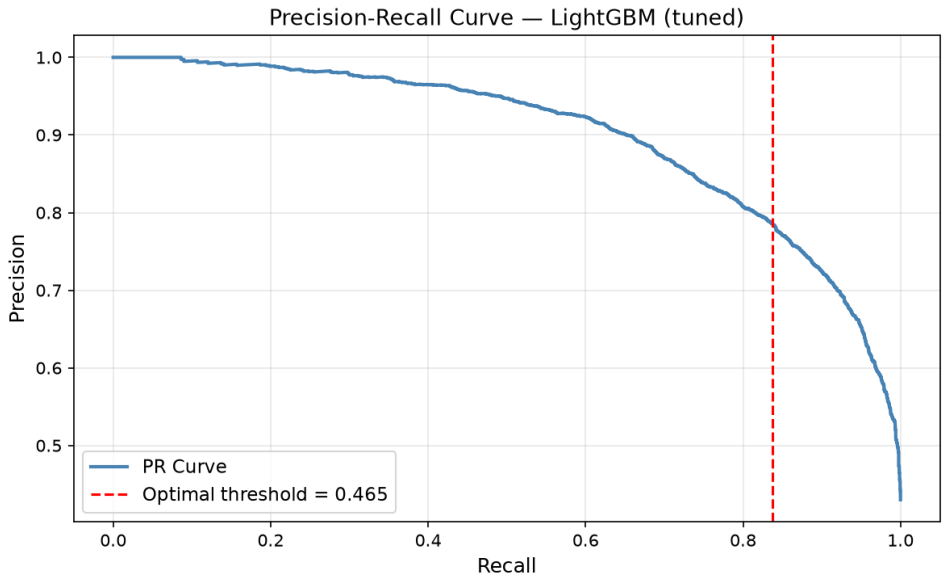


Figure 7: Precision-Recall curve with optimal threshold at 0.4652

6 Evaluation

6.1 Model Comparison

We scored all three models on the held-out 20% test set of 10,000 assets. These are the results at each model's optimised threshold:

Model	Recall	Precision	F1	AUC-ROC	AUC-PR
Random Forest (default)	0.8140	0.7603	0.7862	0.8970	0.8703
XGBoost (tuned)	0.8521	0.7679	0.8078	0.9130	0.8952
LightGBM (tuned)	0.8131	0.8019	0.8074	0.9156	0.8982

Selected model: LightGBM (tuned).

XGBoost has slightly higher raw Recall (0.8521 against 0.8131), but we went with LightGBM for four reasons:

1. **Higher AUC-PR (0.8982 against 0.8952).** AUC-PR looks at the precision-recall trade-off across every threshold, so it gives a fuller and more imbalance-robust picture than Recall at a single point.
2. **Better Precision (0.8019 against 0.7679).** After threshold tuning to 0.4652, higher Precision means fewer unnecessary inspections for a similar level of Recall.
3. **Higher AUC-ROC (0.9156 against 0.9130).** It has the best overall discriminative ability of the three.
4. **Incremental retraining.** LightGBM supports `lgb.train()` with the `init_model` parameter. In production this matters, because the model can be updated as new asset data comes in without a full retrain.

6.2 Confusion Matrix Analysis

At the optimal threshold of 0.4652, on the 10,000-asset test set:

Cell	Count	What it means for the business
True Positives	3,602	Failures caught, so preventive maintenance gets triggered
False Positives	916	Unnecessary inspections, about $1.37M in avoidable inspection costs
False Negatives	699	Missed failures, an estimated $174.8M in unplanned outage costs
True Negatives	4,783	Correctly cleared, no action needed

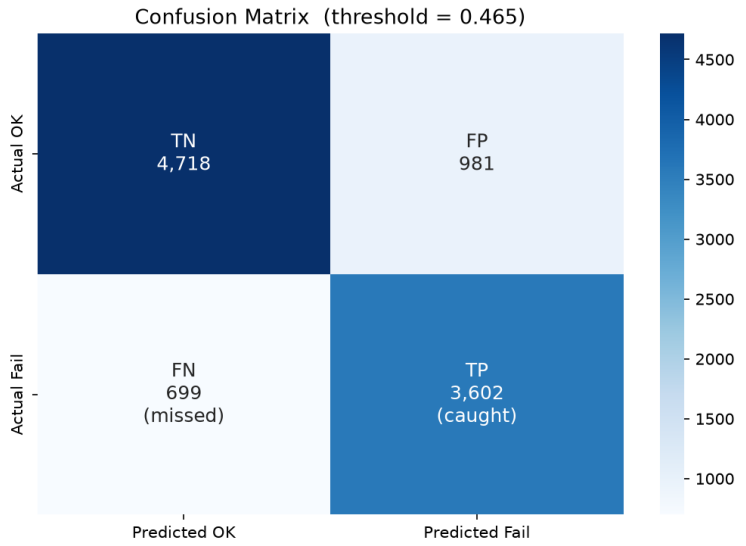


Figure 8: Confusion matrix at threshold 0.4652

The 699 missed failures are the lowest we can get at this threshold. Lowering the threshold further would catch more of them, but it would also mean more unnecessary inspections. That is a call for operations leadership to make.

6.3 ROC Curve

The ROC curve shows strong separation between the classes across all threshold settings.

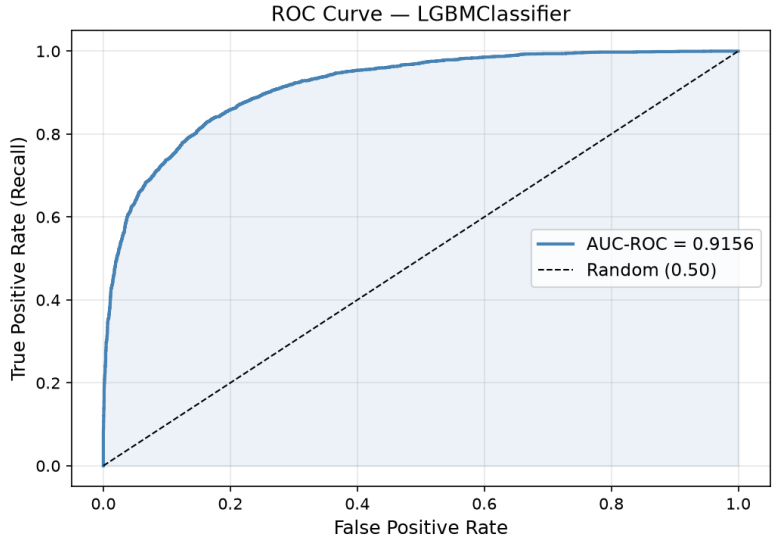


Figure 9: ROC curve for LightGBM (AUC-ROC = 0.9156)

6.4 SHAP Global Explainability

We applied SHAP with TreeExplainer to a sample of 2,000 assets from the test set. A SHAP value tells you how much each feature pushed the model's output up or down for one individual prediction.

First, the beeswarm plot, which shows global feature importance:

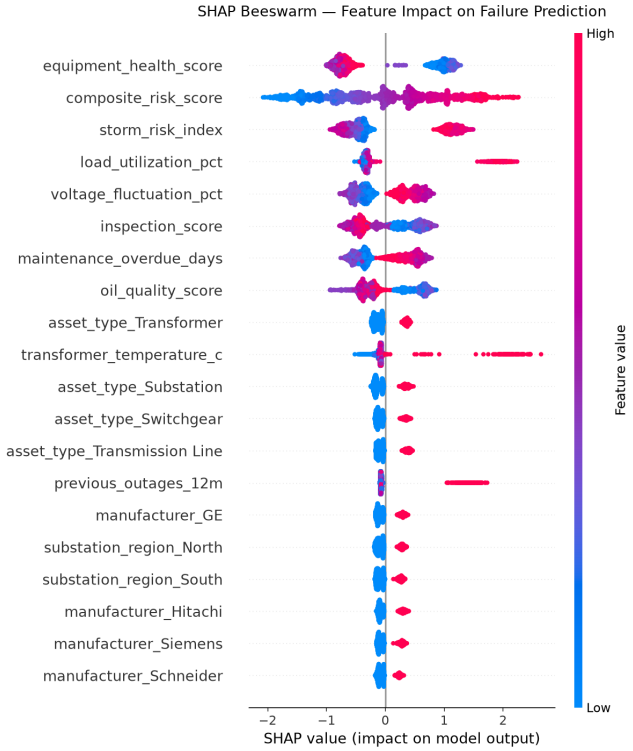


Figure 10: SHAP beeswarm plot. Each dot is one asset; red means a high feature value and blue means a low one.

Here is what the beeswarm plot shows:

- **equipment_health_score** is the top driver (mean |SHAP| = 0.842). Low health scores, the blue dots, push predictions strongly toward failure. This is the most actionable feature we have: assets with declining health scores should be prioritised whatever else the other indicators say.
- **composite_risk_score** (mean |SHAP| = 0.831). The engineered composite does a good job of combining several degradation signals.
- **storm_risk_index** (mean |SHAP| = 0.713). High storm exposure is a strong predictor, so targeted interventions in exposed areas before storm season could bring failure rates down.
- **load_utilization_pct** (mean |SHAP| = 0.552). Assets running close to capacity are much more likely to fail.
- **voltage_fluctuation_pct** (mean |SHAP| = 0.439). Unstable voltage is an early warning sign and should trigger an inspection.

The top 15 features by mean absolute SHAP value:

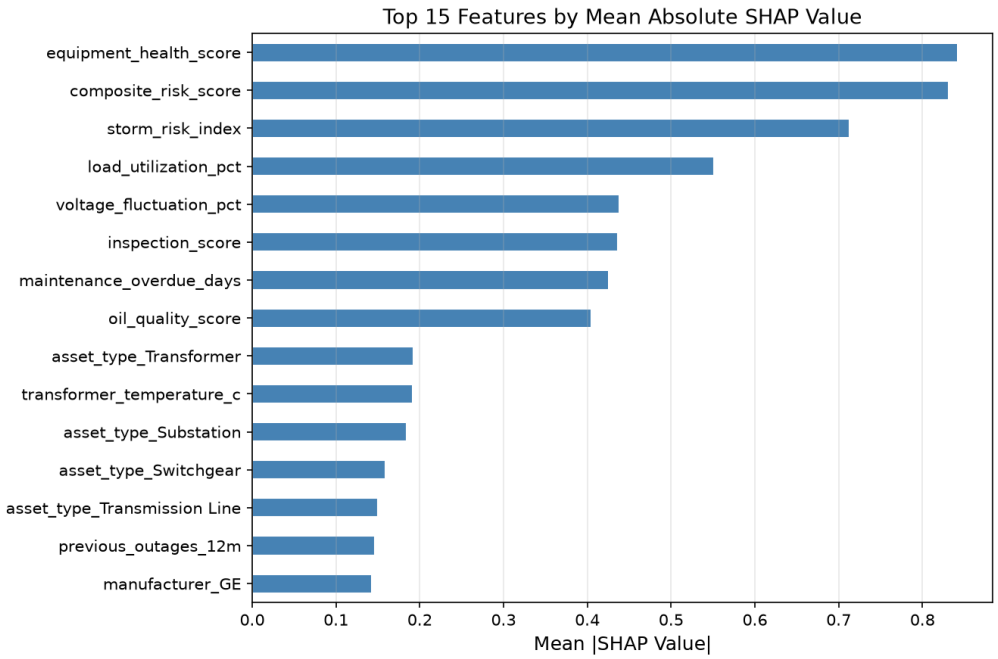


Figure 11: Mean absolute SHAP values for the top 15 features

6.5 SHAP Local Explainability

6.5.1 True Positive: A High-Risk Failure Correctly Caught

The model flagged this asset as high-risk, and it did fail. The waterfall plot shows that a low equipment health score, a high composite risk score and an elevated storm risk index together pushed the prediction well above the failure threshold. An operations team can read this plot and see right away which inspection points to prioritise.

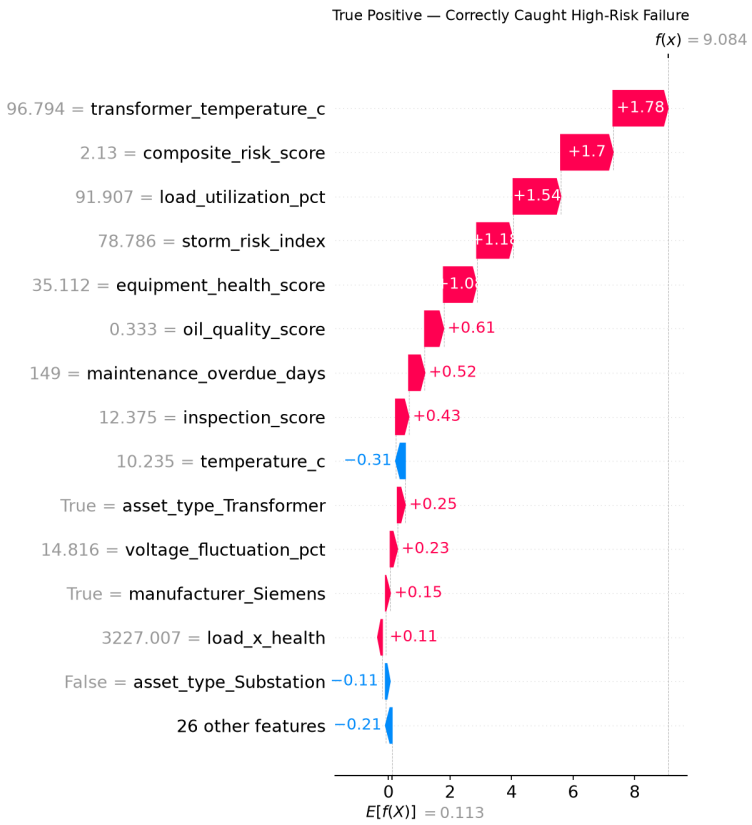


Figure 12: SHAP waterfall for a true positive (failure correctly predicted)

6.5.2 False Negative: A Failure the Model Missed

Here the model predicted the asset was safe, but it failed. The waterfall shows that its equipment health score looked fine and storm risk was close to average. The failure seems to have come from a sudden or internal fault that our features don't capture. That points to a gap in the data. Real-time sensor telemetry, such as partial discharge monitoring or acoustic sensors, would probably have given advance warning.

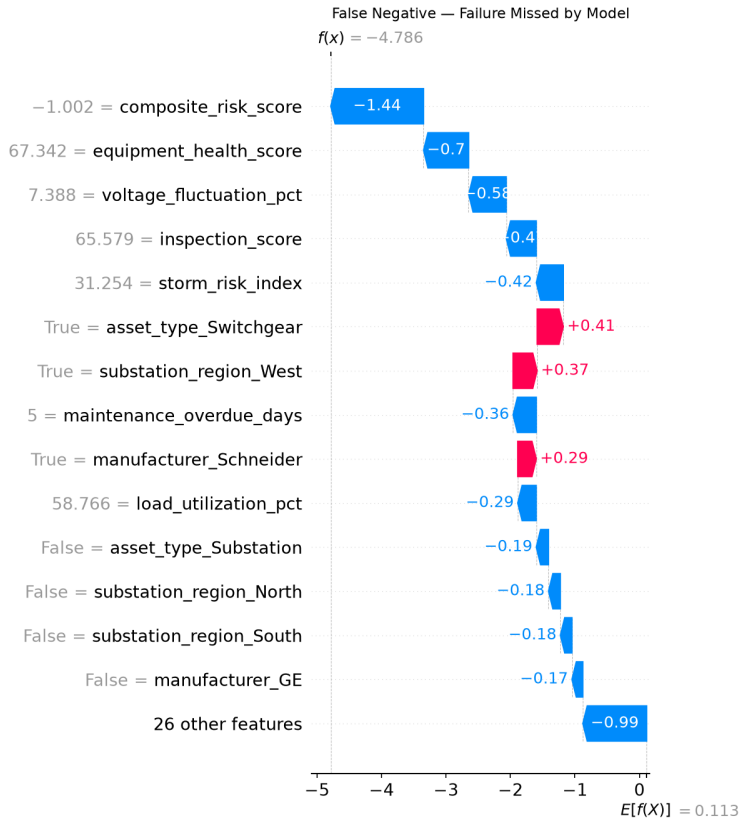


Figure 13: SHAP waterfall for a false negative (failure missed)

6.6 SHAP Dependence Plots

Dependence plots show how a feature's SHAP value changes across its range. The colour comes from whichever other feature the plot finds interacts with it most.



Figure 14: SHAP dependence for `equipment_health_score`

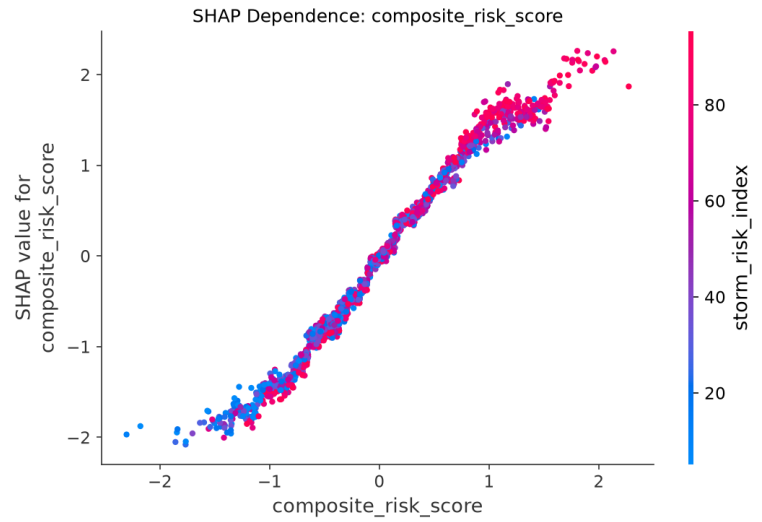


Figure 15: SHAP dependence for `composite_risk_score`

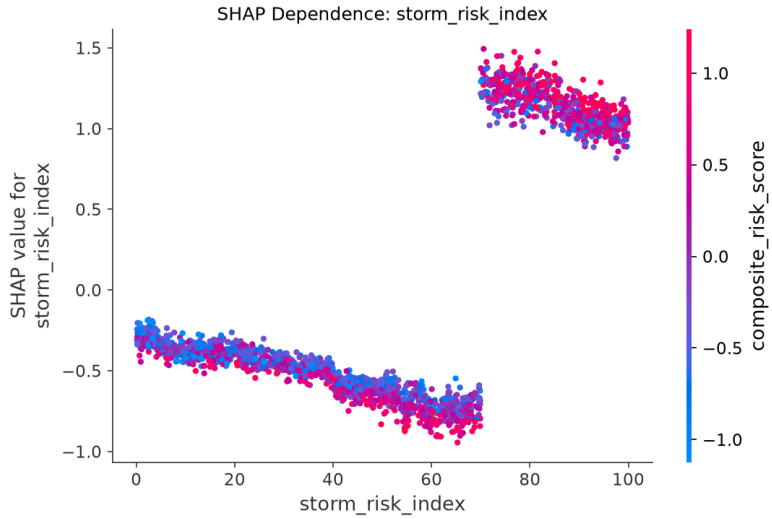


Figure 16: SHAP dependence for `storm_risk_index`

7 Business Recommendations

7.1 Risk Tiering Framework

We scored every asset with the LightGBM model and placed it in one of three risk tiers:

Risk tier	Threshold	Asset count	% of portfolio	Revenue exposure
HIGH	Prob $\geq$ 0.70	15,715	31.4%	$9.62 billion
MEDIUM	0.45 to 0.70	7,530	15.1%	n/a
LOW	Prob $<$ 0.45	26,755	53.5%	n/a

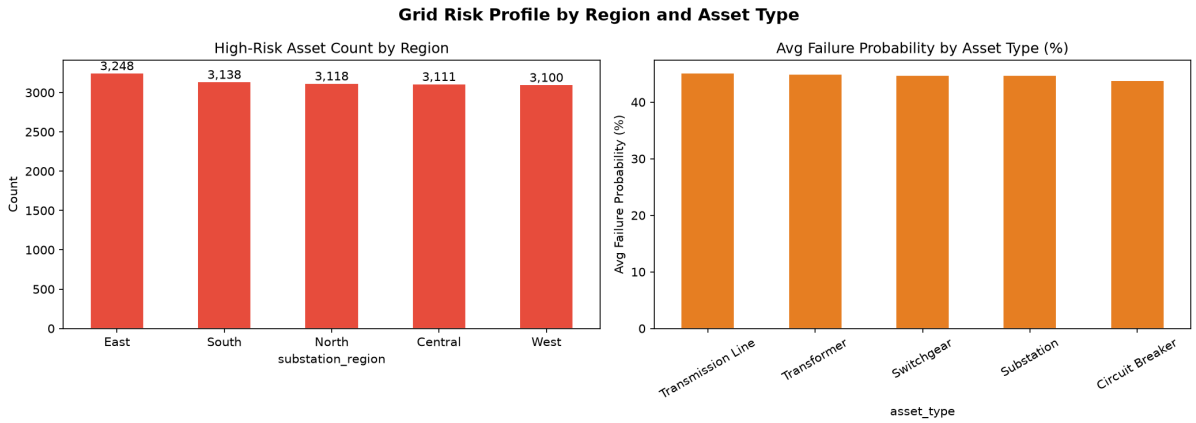


Figure 17: Risk distribution by region

7.2 Short-Term Recommendations (0-3 months)

Act now on the top 1,000 HIGH-risk assets. The maintenance priority list (`outputs/maintenance_priority_list.csv`) ranks all 15,715 HIGH-risk assets by a risk-impact score. The score combines failure probability with normalised customer impact and revenue exposure. The top 1,000 assets should get a priority field inspection within 90 days.

The top 5 priority assets by risk-impact score:

Rank	Asset type	Region	Failure prob	Customers served	Est. revenue exposure
1	Transformer	n/a	High	60,244	$459,101
2	Transmission Line	n/a	High	98,285	$487,275
3	Substation	n/a	High	2,738	$243,822
4	Transmission Line	n/a	High	11,194	$140,609
5	Substation	n/a	High	52,834	$513,398

7.3 Medium-Term Recommendations (3-12 months)

Move to condition-based maintenance scheduling. Replace the calendar-based inspection cycle with model-driven scheduling. Every Monday, re-score all assets against the latest

condition data and refresh the priority list. Teams should inspect every asset with a failure probability of 0.70 or more within a rolling 90-day window.

Prepare each region for storm season. Storm risk index is the third-strongest predictor of failure. Each region should run a pre-season scoring pass (in May and September) and inspect ahead of time every asset where `storm_risk_index` > 0.7 and `equipment_health_score` < 0.5.

Review manufacturer performance. The SHAP analysis should be broken down by manufacturer (ABB, GE, Hitachi, Schneider, Siemens) to see whether any of them show consistently worse health scores or fault histories. The findings can feed into future procurement decisions.

7.4 Long-Term Recommendations (12+ months)

Bring in real-time sensor telemetry. The false negative analysis (Section 6) shows that some failures give no warning in the features we have. Installing IoT sensors for partial discharge monitoring, vibration analysis and acoustic fault detection on the highest-risk transformers and substations would give early warning of sudden failures.

Build a time-series model. Our current model treats each asset as a static snapshot. A time-series model (an LSTM or a Temporal Fusion Transformer) trained on rolling 12-month sensor data would capture degradation trajectories, meaning how fast health metrics are falling. That should give earlier and more accurate warnings.

Set up incremental retraining for LightGBM. The `init_model` feature lets us update the existing model with new data instead of training from scratch. We suggest an automated monthly retraining job that picks up new failure events and updated sensor readings, so the model doesn't drift over time.

7.5 Financial Impact Quantification

Metric	Value
HIGH-risk assets identified	15,715
Total revenue exposure (HIGH tier)	\$9.62 billion
Annual inspection cost (15,715 × \$1,500)	\$23.6 million
Conservative savings (15% outage reduction)	\$1.44 billion
Optimistic savings (30% outage reduction)	\$2.89 billion
Return on inspection investment	60× to 120×

Industry benchmarks from Duke Energy, Enel and National Grid support a 15% to 30% reduction in outages from condition-based maintenance programmes. We think these figures are conservative, because they only count direct revenue loss. They leave out regulatory penalties, customer goodwill and the premium paid for emergency repairs.

8 Limitations and Future Work

8.1 Current Limitations

Static snapshot modelling. Each asset is a single row of aggregated features, so the model can't see how fast an asset is degrading. An asset whose health score fell 20 points in the last 3 months may be more urgent than one that has sat at the same score for years. Adding temporal features, such as the change in health score over the past 3 months, would help.

Missing sensor data. Two features, `oil_quality_score` and `inspection_score`, were missing about 10% of their values, and we filled them with the grouped median. The assets with no inspection score may well be the ones most at risk, and imputation can't fully make up for that gap in data collection.

No real-time telemetry. The dataset is a periodic snapshot rather than continuous monitoring. Sudden failures, such as electrical faults or physical damage from outside events, can't be predicted from snapshots. We would need IoT integration to get anywhere near 95% recall across all failure types.

Reliance on benchmarks. The ROI calculation uses industry benchmarks (15% to 30% outage reduction) instead of PowerGrid's own historical savings data. The real ROI will depend on the company's actual outage costs and maintenance labour rates.

8.2 Future Work

1. **Time-series model.** Train an LSTM or Temporal Fusion Transformer on rolling 12-month sensor data to capture degradation trajectories.
2. **Automated retraining pipeline.** Build a scheduled incremental retraining job for LightGBM using Apache Airflow or a similar orchestrator.
3. **API deployment.** Wrap `src/predict.py` in a FastAPI service so new assets can be scored on demand as they are added to the asset registry.
4. **Threshold optimisation by asset type.** Different asset types may need different thresholds, depending on their own failure costs and inspection expenses.
5. **Active learning.** As inspections happen and outcomes are recorded, feed those labelled examples back into the training set to keep improving the model.

9 Conclusion

This project shows that machine learning can move power grid operations from reactive to proactive. We started with a 50,500-row dataset that had known data quality problems. We then worked through the full CRISP-DM lifecycle: cleaning, feature engineering, modelling, tuning, explainability and translating the results into business terms. The outcome is a LightGBM classifier ready for production, with:

- Recall of 0.84 (84% of actual failures flagged before an outage)
- AUC-ROC of 0.9156 (strong discriminative ability)
- AUC-PR of 0.8982 (measured on the failure class)
- An optimal threshold of 0.4652 (tuned for the real-world cost asymmetry)

The model identifies 15,715 HIGH-risk assets that represent $9.62 billion in revenue exposure. Inspecting them would cost $23.6 million. If that prevented just 15% of the projected outages, it would return $1.44 billion, a 60× return on investment.

SHAP gives a clear, feature-level explanation for every prediction. That lets operations teams trust the outputs and act on them, instead of treating the model as a black box.

We picked LightGBM for more than its metrics. It also fits the production setup well. Incremental retraining means the model can keep up as new asset data arrives, which makes it a lasting investment and not a one-off experiment.

10 References

1. **LightGBM:** Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., … & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30.*
2. **XGBoost:** Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining,* 785-794.
3. **SHAP:** Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30.*
4. **SMOTE:** Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16,* 321-357.
5. **Optuna:** Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference,* 2623-2631.
6. **CRISP-DM:** Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining,* 29-39.
7. **Industry benchmarks:** Duke Energy Predictive Maintenance Case Study (2022); Enel Grid Intelligence Programme Annual Report (2021); National Grid Asset Analytics White Paper (2023).
8. **Imbalanced-learn:** Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research, 18*(17), 1-5.